

SKILL-6

SESSION-11

Task-1: To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];
int history_count = 0;

/* Store a command in the history buffer */
void store_command(const char *command)
{
    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        /* Shift old commands to make space */
        for (int i = 1; i < MAX_HISTORY; i++)
        {
            strcpy(history[i - 1], history[i]);
        }
        strcpy(history[MAX_HISTORY - 1], command);
    }
}

/* Display all stored commands */
void display_history()
{
    printf("\nCommand History\n");
    printf("-----\n");

    if (history_count == 0)
    {

```

```
^G Help          ^O Write Out     ^W Where Is      ^K Cut
^Y Exit          ^P Read File     ^_ Replace       ^U Paste
```

```

GNU nano 7.2
{
    printf("History is empty.\n");
    return;
}

for (int i = 0; i < history_count; i++)
{
    printf("%d  %s\n", i + 1, history[i]);
}
}

/* Retrieve a particular history entry */
void retrieve_entry(int number)
{
    if (number < 1 || number > history_count)
    {
        printf("Invalid history entry.\n");
        return;
    }

    printf("History entry %d: %s\n", number, history[number - 1]);
}

/* Validate history consistency */
void validate_history()
{
    if (history_count < 0 || history_count > MAX_HISTORY)
    {
        printf("History consistency check: FAILED\n");
        return;
    }

    printf("History consistency check: PASSED\n");
    printf("Stored entries: %d/%d\n", history_count, MAX_HISTORY);
}

int main()

```

```

GNU nano 7.2
{
    char command[MAX_COMMAND];
    int choice;
    int number;

    printf("Command History Buffer\n");
    printf("-----\n");
    printf("Maximum history size: %d\n", MAX_HISTORY);

    while (1)
    {
        printf("\n1. Store Command\n");
        printf("2. Display History\n");
        printf("3. Retrieve Entry\n");
        printf("4. Validate History\n");
        printf("5. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();

        switch (choice)
        {
            case 1:
                printf("Enter command: ");
                fgets(command, sizeof(command), stdin);

                command[strcspn(command, "\n")] = '\0';

                store_command(command);

                printf("Command stored successfully.\n");
                break;

            case 2:
                display_history();
                break;

```

```

GNU nano 7.2
printf("Enter command: ");
fgets(command, sizeof(command), stdin);

command[strcspn(command, "\n")] = '\0';

store_command(command);

printf("Command stored successfully.\n");
break;

case 2:
display_history();
break;

case 3:
printf("Enter history entry number: ");
scanf("%d", &number);

retrieve_entry(number);
break;

case 4:
validate_history();
break;

case 5:
printf("Exiting history manager...\n");
return 0;

default:
printf("Invalid choice.\n");

}

return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano skill6s11task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill6s11task1.c -o skill6s11task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill6s11task1
Command History Buffer
-----
Maximum history size: 5

1. Store Command
2. Display History
3. Retrieve Entry
4. Validate History
5. Exit
Enter choice: 1
Enter command: ls
Command stored successfully.

1. Store Command
2. Display History
3. Retrieve Entry
4. Validate History
5. Exit
Enter choice: 1
Enter command: pwd
Command stored successfully.

1. Store Command
2. Display History
3. Retrieve Entry
4. Validate History
5. Exit
Enter choice: 1
Enter command: cd /tmp
Command stored successfully.

1. Store Command
2. Display History
3. Retrieve Entry
4. Validate History
5. Exit
Enter choice: 1
Enter command: Command stored successfully.

```

```

Command History
-----
1  ls
2  pwd
3  cd /tmp
4
5  who

1. Store Command
2. Display History
3. Retrieve Entry
4. Validate History
5. Exit
Enter choice: 3
Enter history entry number: 3
History entry 3: cd /tmp

1. Store Command
2. Display History
3. Retrieve Entry
4. Validate History
5. Exit
Enter choice: 1
Enter command: echo Hello
Command stored successfully.

1. Store Command
2. Display History
3. Retrieve Entry
4. Validate History
5. Exit
Enter choice: 3
Enter history entry number: 10
Invalid history entry.

1. Store Command
2. Display History
3. Retrieve Entry
4. Validate History
5. Exit
Enter choice: |

```



Observation

The program successfully implements a command history buffer using a fixed-size array. Commands are stored and displayed with their corresponding history numbers. When the buffer reaches its maximum capacity of five commands, the oldest command is removed to make space for the new command. Individual history entries can be retrieved using their entry number. Invalid entry numbers are handled appropriately, and the consistency of the history buffer is verified successfully.

Task-2: To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 3
#define MAX_INPUT 200

/* Store pipeline commands */
char commands[MAX_COMMANDS][MAX_INPUT];
int command_count = 0;

/* Display the pipeline layout */
void display_pipeline()
{
    printf("\nPipeline Layout\n");
    printf("-----\n");

    if (command_count == 0)
    {
        printf("No commands in pipeline.\n");
        return;
    }

    for (int i = 0; i < command_count; i++)
    {
        printf("[%d] %s", i + 1, commands[i]);

        if (i < command_count - 1)
            printf(" --> ");

        printf("\n");
    }

    printf("\n");
}

/* Validate pipeline structure */

```

⌘G Help
⌘O Write Out
⌘W Where Is
⌘K Cu
⌘X Exit
⌘R Read File
⌘\ Replace
⌘U Pa

```

        close(fd[1]);

        waitpid(pid1, NULL, 0);
        waitpid(pid2, NULL, 0);
    }

/* Execute a three-command pipeline */
void execute_three_commands()
{
    int pipe1[2];
    int pipe2[2];

    pid_t pid1, pid2, pid3;

    if (pipe(pipe1) == -1 || pipe(pipe2) == -1)
    {
        perror("pipe");
        return;
    }

    /* First command */
    pid1 = fork();

    if (pid1 == 0)
    {
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp(commands[0], commands[0], NULL);

        perror("First command failed");
        exit(1);
    }
}

```

```

        perror("Second command failed");
        exit(1);
    }

    /* Third command */
    pid3 = fork();
    if (pid3 == 0)
    {
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp(commands[2], commands[2], NULL);

        perror("Third command failed");
        exit(1);
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);
    waitpid(pid3, NULL, 0);
}

int main()
{
    int choice;

    printf("Pipeline Structure and Execution\n");

```

^G Help
^O Write Out
^W Where Is
^K Cut
^T Exe
^X Exit
^R Read File
^_ Replace
^U Paste
^J Jus

```

        break;

    case 2:
        display_pipeline();
        break;

    case 3:
        validate_pipeline();
        break;

    case 4:
        if (command_count == 2)
        {
            execute_two_commands();
        }
        else if (command_count == 3)
        {
            execute_three_commands();
        }
        else
        {
            printf("Invalid pipeline configuration.\n");
        }
        break;

    case 5:
        printf("Exiting pipeline manager...\n");
        return 0;

    default:
        printf("Invalid choice.\n");
    }
}

return 0;
}

```

^G Help
^O Write Out
^W Where Is
^K Cut
^X Exit
^R Read File
^_ Replace
^U Paste

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano skill6s11task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill6s11task2.c -o skill6s11task2.c
gcc: fatal error: input file 'skill6s11task2.c' is the same as output file
compilation terminated.
ruhi0404@Ruhitha:~/Operating_Systems$
ruhi0404@Ruhitha:~/Operating_Systems$
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill6s11task2.c -o skill6s11task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill6s11task2
Pipeline Structure and Execution
-----
1. Add Commands
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 1
Enter number of commands (2 or 3): 2
Enter command 1:
Enter command 2: wc
Commands stored successfully.

1. Add Commands
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 1
Enter number of commands (2 or 3): 2
Enter command 1: ls
Enter command 2: wc
Commands stored successfully.

1. Add Commands
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 2

Pipeline Layout
-----
[1] ls --> [2] wc

```

```

1. Add Commands
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: exit
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
0 0 0

1. Add Commands
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
0 0 0

Enter choice:
1. Add Commands
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
0 0 0

Enter choice:
1. Add Commands
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Usage: grep [OPTION]... PATTERNS [FILE]...
Try 'grep --help' for more information.
0 0 0

Enter choice:
1. Add Commands
2. Display Pipeline
3. Validate Pipeline
4. Execute Pipeline
5. Exit
Enter choice: 5
Exiting pipeline manager...
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully creates and maintains a pipeline structure for two or three commands. Commands are stored in their execution order and the pipeline layout is displayed for verification. Pipes are created using `pipe()` and process input/output is connected using `dup2()`. Multiple child processes are created using `fork()` and synchronized using `waitpid()`. Both two-command and three-command pipeline configurations were tested, and invalid pipeline configurations were detected.

SESSION-12

Task-1: To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t pid1, pid2;

    printf("Pipe Communication and Data Flow\n");
    printf("-----\n");

    /* Create pipe */
    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipe created successfully.\n");

    /* Create first child */
    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        /* Child 1: execute ls and send output to pipe */
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);

        execlp("ls", "ls", NULL);
    }
}

```

```

        perror("execlp ls");
        exit(1);
    }

    /* Create second child */
    pid2 = fork();

    if (pid2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid2 == 0)
    {
        /* Child 2: receive pipe data and execute wc */
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);

        execlp("wc", "wc", "-l", NULL);

        perror("execlp wc");
        exit(1);
    }

    /* Parent closes both descriptors */
    close(fd[0]);
    close(fd[1]);

    /* Synchronize child processes */
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Data flow completed successfully.\n");
    printf("Child processes synchronized.\n");

    return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$
ruhi0404@Ruhitha:~/Operating_Systems$ nano skill6s12task1.c
ruhi0404@Ruhitha:~/Operating_Systems$
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill6s12task1.c -o skill6s12task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill6s12task1
Pipe Communication and Data Flow
-----
Pipe created successfully.
89
Data flow completed successfully.
Child processes synchronized.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully creates an unnamed pipe for inter-process communication. The first child process executes `ls` and redirects its standard output to the pipe. The second child process redirects its standard input from the pipe and executes `wc -l`. Unused file descriptors are closed in the parent and child processes. The parent process uses `waitpid()` to synchronize with both child processes. The successful output demonstrates that data flows correctly from the first process through the pipe to the second process.

Task-2: To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NUM_COMMANDS 4
#define NUM_PIPES 3

int main()
{
    int pipes[NUM_PIPES][2];
    pid_t pids[NUM_COMMANDS];

    char *commands[NUM_COMMANDS] = {
        "ls",
        "sort",
        "uniq",
        "wc"
    };

    printf("Multiple Pipe Pipeline Execution\n");
    printf("-----\n");
    printf("Pipeline: ls | sort | uniq | wc\n\n");

    /* Create multiple pipes */
    for (int i = 0; i < NUM_PIPES; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            return 1;
        }

        printf("Pipe %d created successfully.\n", i + 1);
    }

    /* Create multiple child processes */
    for (int i = 0; i < NUM_COMMANDS; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)

```

```

if (pids[i] == 0)
{
    /* First command */
    if (i == 0)
    {
        dup2(pipes[0][1], STDOUT_FILENO);
    }

    /* Middle commands */
    else if (i > 0 && i < NUM_COMMANDS - 1)
    {
        dup2(pipes[i - 1][0], STDIN_FILENO);
        dup2(pipes[i][1], STDOUT_FILENO);
    }

    /* Last command */
    else
    {
        dup2(pipes[i - 1][0], STDIN_FILENO);
    }

    /* Close all pipe descriptors in child */
    for (int j = 0; j < NUM_PIPES; j++)
    {
        close(pipes[j][0]);
        close(pipes[j][1]);
    }

    /* Execute command */
    if (i == 0)
    {
        execlp("ls", "ls", NULL);
    }
    else if (i == 1)
    {
        execlp("sort", "sort", NULL);
    }
    else if (i == 2)
    {
        execlp("uniq", "uniq", NULL);
    }
}

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]W Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

```

        {
            execlp("wc", "wc", "-l", NULL);
        }

        perror("Command execution failed");
        exit(1);
    }
}

/* Parent closes all pipe descriptors */
for (int i = 0; i < NUM_PIPES; i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

printf("All processes launched successfully.\n");

/* Wait for all child processes */
for (int i = 0; i < NUM_COMMANDS; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("All processes completed successfully.\n");
printf("Pipeline resources cleaned up.\n");

return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano skill6s12task2.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill6s12task2.c -o skill6s12task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill6s12task2
Multiple Pipe Pipeline Execution
-----
Pipeline: ls | sort | uniq | wc

Pipe 1 created successfully.
Pipe 2 created successfully.
Pipe 3 created successfully.
All processes launched successfully.
91
All processes completed successfully.
Pipeline resources cleaned up.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

Observation

The program successfully creates and manages multiple pipes using a two-dimensional file-descriptor array. Four child processes are launched to execute the commands `ls`, `sort`, `uniq`, and `wc -l`. The pipes connect the standard output of one process to the standard input of the next process, forming a long pipeline. All unused file descriptors are closed after the required connections are established. The parent process waits for all child processes using `waitpid()`, ensuring coordinated execution and proper cleanup.

